

DIPLOMATURA EN IA - UNIVERSIDAD AUSTRAL — TRABAJO PRÁCTICO MÓDULO 1

Predicción del puntaje de vinos

Informe generado a partir de la mejor ejecución registrada de cada modelo y de la ejecución de comparación cabeza a cabeza en Weights & Biases. Autocontenido: imágenes y tablas incrustadas, no se necesita una cuenta de W&B para leerlo.

github.com/leonardoheis/diplo-mod-1

leonardoheis.github.io/diplo-mod-1

CONTENIDO

3.1 Introducción - El dataset y el problema

3.2 Análisis exploratorio (EDA) - Qué limita el dataset

3.3 Preprocesamiento - Limpieza, codificación y split

3.4 Entrenamiento

XGBoost

Red Neuronal

3.5 Evaluación y comparación

RMSE / MAE / R^2 por partición

Tarjeta de puntuación

Sobreajuste por modelo

Predicho vs. real

Error por rango de puntuación

Superposición de residuos

¿Qué atributos importan más? - Importancia de atributos (SHAP)

¿Cuál modelo y por qué? - XGBoost vs. Red Neuronal

¿Hay overfitting? - El train R^2 es más alto que el de test

Ejemplo aplicado - Predicción de un vino nuevo

¿Qué mejoraría con más tiempo o datos? - Modelado

¿Qué mejoraría con más tiempo o datos? - Datos

3.6 Conclusiones

Lecciones aprendidas

Síntesis final

3.1 INTRODUCCIÓN

El dataset y el problema

Wine Reviews (Wine Enthusiast, vía Kaggle): 129.971 reseñas de vino, 14 columnas originales — país, descripción de cata, precio, provincia, región, catador, variedad, bodega, título (de donde se extrae el año de añada) — más la variable objetivo `points`.

Variable objetivo: `points`, el puntaje que el catador le dio al vino, en una escala continua de **80 a 100**. Es un problema de **regresión**, no de clasificación: se predice un puntaje numérico, no una categoría.

Pipeline completo: EDA (`01`) — preprocesamiento (`02`) — dos modelos entrenados sobre el mismo conjunto de atributos, **XGBoost** (`03`) y una **red neuronal en PyTorch** (`04`) — evaluación y comparación (`05`).

3.2 ANÁLISIS EXPLORATORIO (EDA)

Qué limita el dataset

Ninguna mejora de modelo o de búsqueda de hiperparámetros elimina estos límites — están en los datos, no en el modelo:

- **Sesgo del catador.** Un ANOVA de un factor sobre `points` por `taster_name` da $F=612.05$, $p \approx 0$, $\eta^2=0.099$ — la identidad del catador explica por sí sola ~10% de la varianza del objetivo.
- **Precio.** 6.9% de las filas no tienen `price` (imputado por mediana agrupada); la distribución cruda tiene asimetría 18.0 y una cola de hasta \$3.300.
- `region_2`. 61.1% de datos faltantes, de forma estructural: solo existe para vinos de EE.UU. y en la mayoría de los casos no es recuperable a partir de `region_1`.
- **Sesgo de supervivencia en la añada.** Los vinos anteriores a 1980 puntúan por encima del promedio no porque sean mejores, sino porque solo las botellas excepcionales siguen existiendo (y siendo reseñadas) décadas después.
- **Categorías de cola larga.** `winery` tiene 16.757 valores únicos, la mayoría con solo un puñado de reseñas.
- **Desbalance en reseñas por vino.** El 91% de los 118.840 vinos únicos (por `title`) tiene una sola reseña, mientras que un puñado llega a acumular hasta 11 — un desbalance real en cuánta evidencia respalda cada puntaje. El 20% de las reseñas, además, no tiene catador identificado (`taster_name` vacío), proporción similar tanto para vinos con una sola reseña como para los que tienen varias.
- **Una sola fuente.** Todo el dataset proviene del estilo editorial de una sola publicación (Wine Enthusiast).

3.3 PREPROCESAMIENTO

Limpieza, codificación y split

Nulos. `region_2` se descarta (61% faltante, estructural). `price` se imputa por mediana agrupada (país, variedad), con respaldo global. `taster_name` / `region_1` faltantes se tratan como categoría propia (los encoders de objetivo devuelven la media global para categorías no vistas).

Codificación categórica. *Target encoding* (con CV de 5 folds al ajustar) para `taster_name`, `variety`, `country`, `region_1`, `winery`, `province`. *Frequency encoding* (log1p de conteos) para `winery`, `variety`, `region_1`. Texto libre (`description`) — TF-IDF, 2000 términos.

Normalización. `StandardScaler` sobre los atributos tabulares, solo para la red neuronal (XGBoost, basado en árboles, es invariante a la escala).

Split y balanceo. 129.971 filas — train 83.180 (64%) / val 20.796 (16%) / test 25.995 (20%), `random_state` fijo. Sin técnicas de balanceo: es un problema de regresión, no de clasificación, no hay clases que desbalancear.

3.4 ENTRENAMIENTO

XGBoost

Por qué XGBoost: gradient boosting sobre árboles es el estándar de facto para datos tabulares de esta escala (~130k filas, atributos mixtos tabulares + bolsa de palabras dispersa), y es notoriamente difícil de superar en este régimen incluso frente a una red neuronal.

Búsqueda de hiperparámetros: Optuna, 50 trials, sobre `max_depth`, `learning_rate`, `n_estimators`, `subsample`, `colsample_bytree`, `min_child_weight`, `reg_alpha`, `reg_lambda`, `gamma`. Early stopping a 20 rondas sin mejora en validación.

HIPERPARÁMETRO

VALOR ELEGIDO

max_depth	7
learning_rate	0.1377
n_estimators	486
subsample	0.971
colsample_bytree	0.878
min_child_weight	5
reg_alpha	0.00068
reg_lambda	2.49e-08
gamma	0.00035

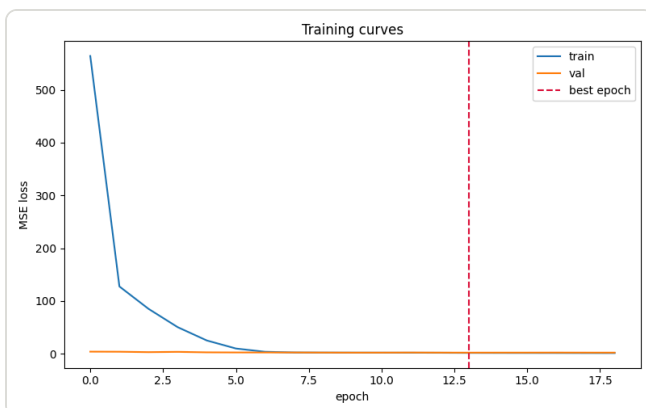
Red Neuronal

Arquitectura: MLP totalmente conectado, capas ocultas `512_128_32`, activación `silu`, dropout 0.435, con BatchNorm1d entre capas.

Regularización: dropout, weight decay, BatchNorm, early stopping, clipping de gradientes y un scheduler `ReduceLROnPlateau` sobre el learning rate.

Optimizador: AdamW, learning rate 0.02701, weight decay 0.002474, batch size 128. Máximo 30 épocas, early stopping con paciencia 5.

Búsqueda: Optuna, 30 trials, sobre `dropout`, `activation`, `batch_size`, `architecture`, `weight_decay`, `learning_rate`.



La pérdida (MSE) de entrenamiento baja de forma pronunciada en las primeras épocas, mientras que la de validación se aplana casi de inmediato. La línea vertical marca la época con mejor RMSE de validación — el punto en el que el early stopping detiene el entrenamiento antes de que el modelo siga sobreajustando al train.

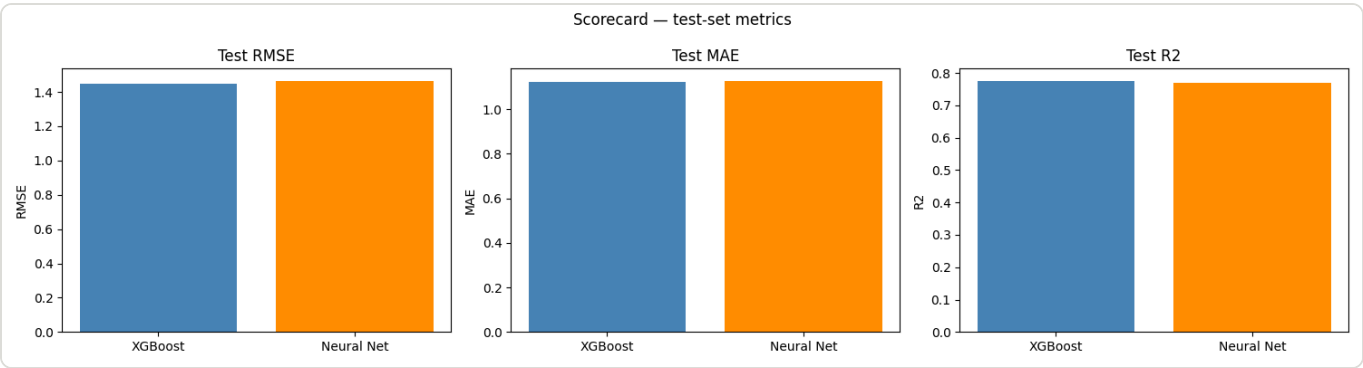
3.5 EVALUACIÓN Y COMPARACIÓN

RMSE / MAE / R² por partición

Ejecución: [comparison-20260801T012442Z](#)

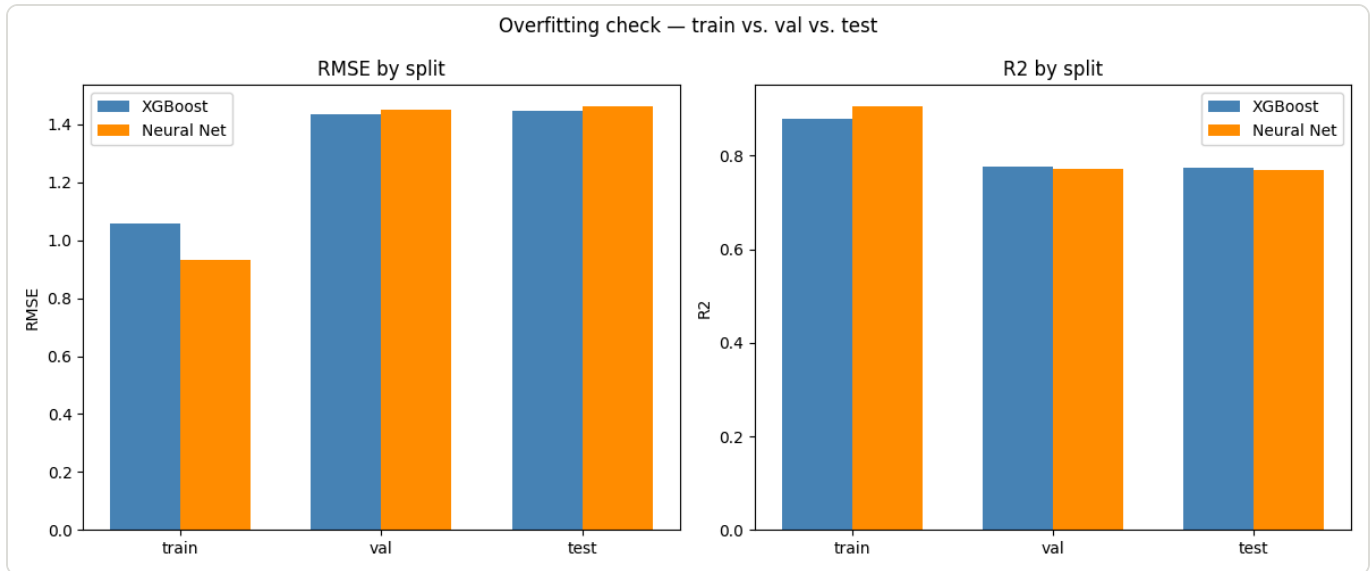
MODEL	RUN_ID	TRAIN_RMSE	TRAIN_MAE	TRAIN_R2	VAL_RMSE	VAL_MAE	VAL_R2	TEST_RMSE	TEST_MAE	TEST_R2
XGBoost	xgboost_tu...	1.0595	0.8341	0.8784	1.4335	1.1104	0.7780	1.4453	1.1203	0.7745
Neural Net	nn_trainin...	0.9319	0.7181	0.9059	1.4494	1.1177	0.7731	1.4627	1.1252	0.7690

Tarjeta de puntuación



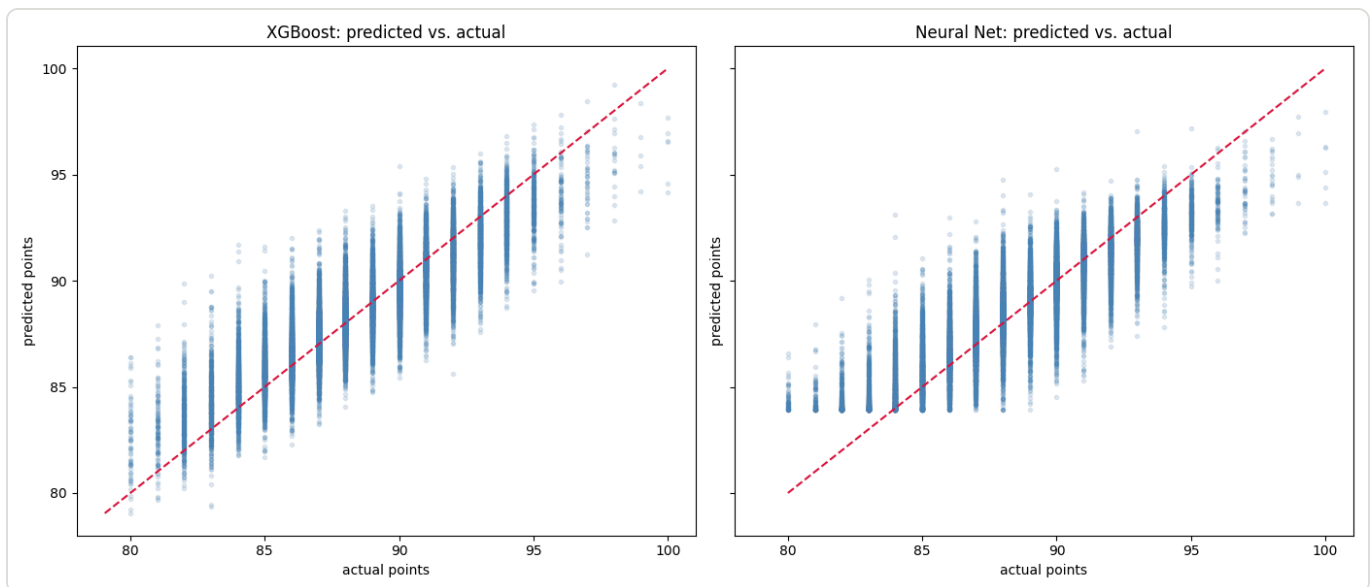
Compara los tres indicadores clave sobre el conjunto de test: XGBoost queda apenas por debajo en RMSE y MAE (menor es mejor) y por encima en R² (mayor es mejor). La diferencia es consistente pero pequeña en las tres métricas — coincide con la brecha de ~1.2% de RMSE relativo entre ambos modelos.

Sobreajuste por modelo



RMSE y R^2 desglosados por partición (train/val/test) para cada modelo. La red neuronal ajusta el train notablemente mejor que XGBoost (RMSE de train más bajo, R^2 de train más alto), pero esa ventaja se reduce casi por completo en val y test — la brecha visual entre las barras de train y las de val/test es, precisamente, el sobreajuste que se discute en la próxima diapositiva.

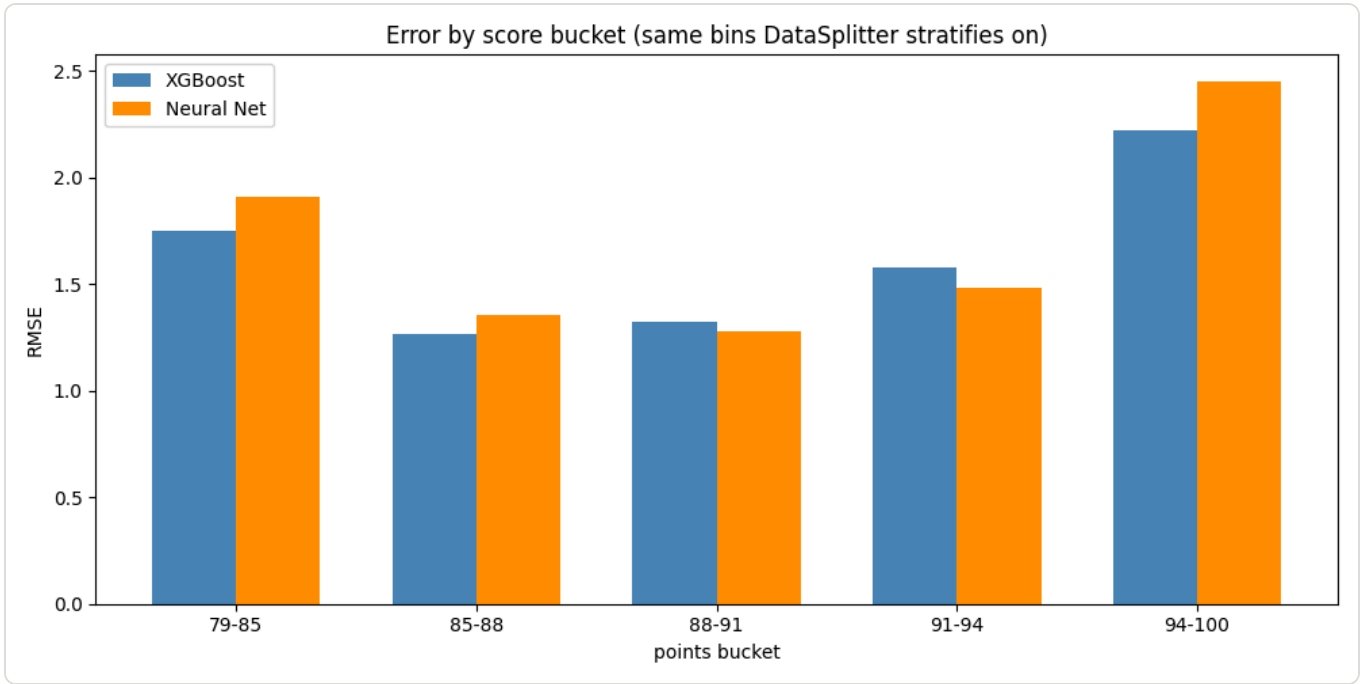
Predicho vs. real



Cada punto es un vino del conjunto de test: el eje X es el puntaje real, el eje Y el puntaje predicho, y la diagonal marca una predicción perfecta. Ambos modelos tienden a comprimir sus predicciones hacia el centro de la escala — sobreestiman los vinos de puntaje muy bajo y subestiman los de puntaje muy

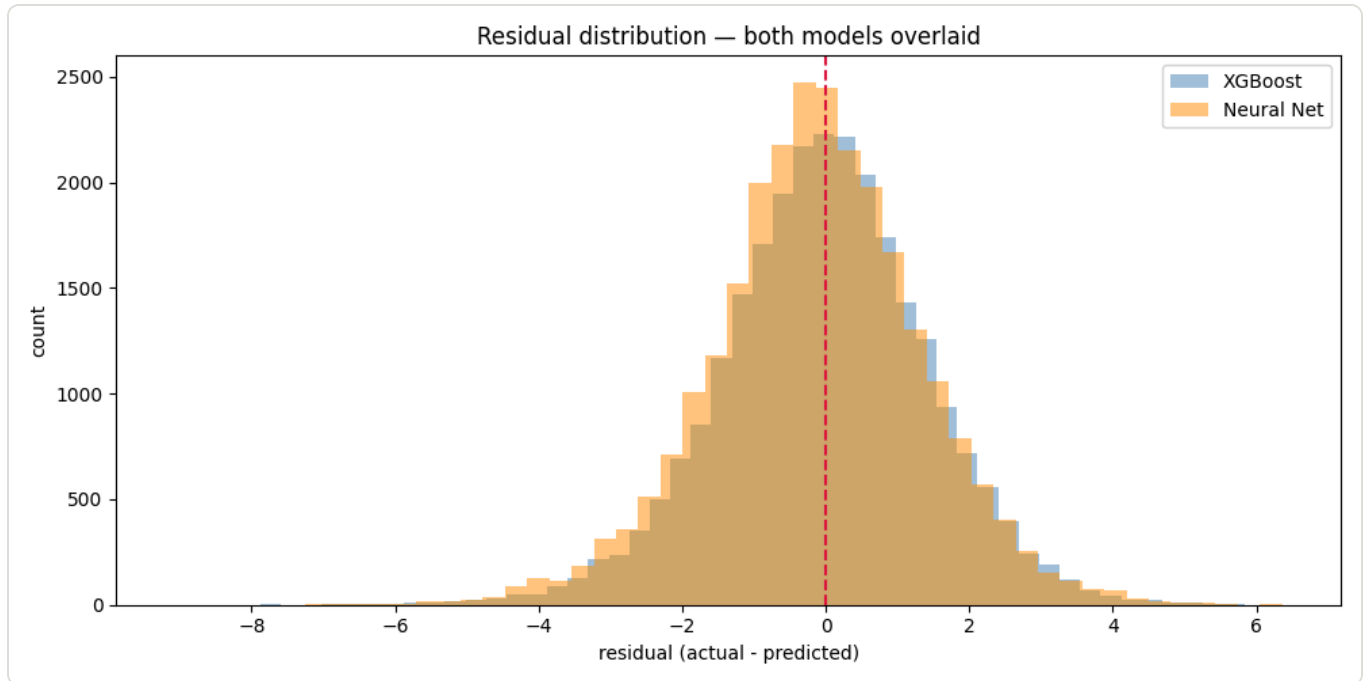
alto — un efecto de regresión a la media típico cuando esos extremos tienen pocos ejemplos de entrenamiento.

Error por rango de puntuación



RMSE calculado por separado dentro de cada rango de puntaje. El error es más bajo en el rango central (85-91, donde se concentra la mayoría de las reseñas) y sube marcadamente en ambos extremos, sobre todo en 94-100 — la misma escasez de datos en los extremos que explica el patrón del gráfico anterior.

Superposición de residuos



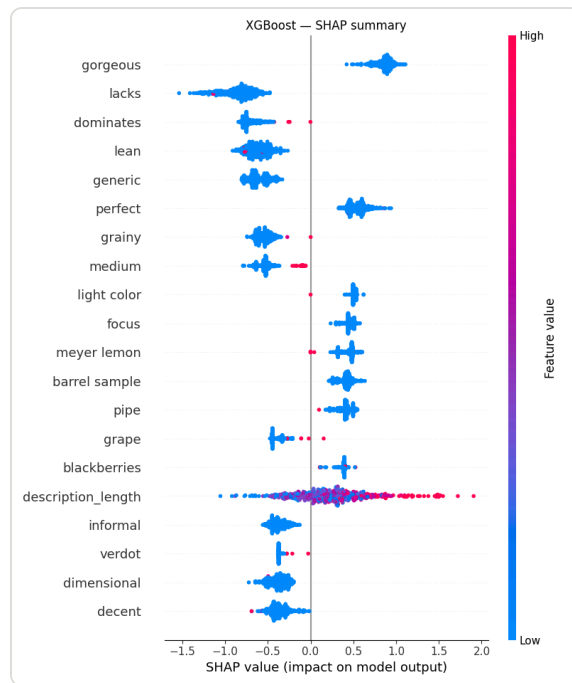
Distribución de los residuos (real menos predicho) de ambos modelos, superpuesta. La forma es aproximadamente simétrica y centrada en cero para los dos — sin sesgo sistemático hacia arriba o abajo — pero el ancho de la distribución es el piso de error que ninguno de los dos modelos logra bajar, coherente con el ruido subjetivo del catador discutido en las conclusiones.

¿QUÉ ATRIBUTOS IMPORTAN MÁS?

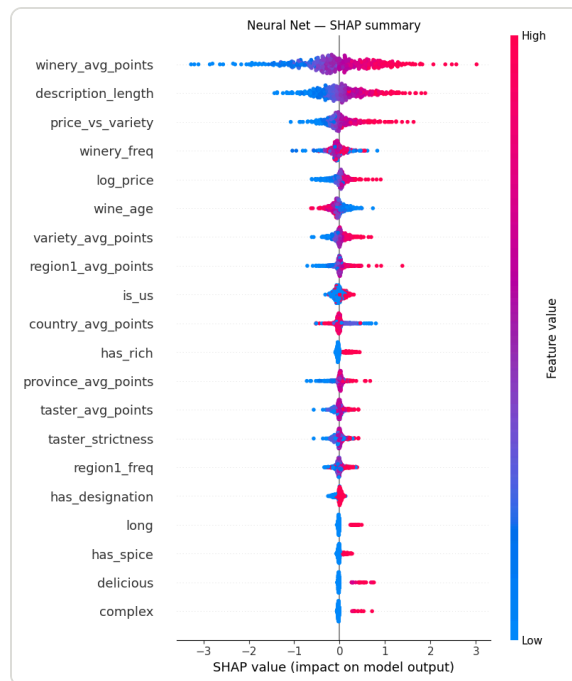
Importancia de atributos (SHAP)

En ambos modelos, `log_price`, `price_vs_variety` y un puñado de términos de texto (TF-IDF) encabezan la importancia SHAP de forma consistente — el precio y el propio texto de la cata son las señales más fuertes, por encima de `region_1`, `variety_avg_points` y `winery_avg_points`.

XGBOOST



RED NEURONAL



¿CUÁL MODELO Y POR QUÉ?

XGBoost vs. Red Neuronal

XGBoost es el mejor modelo aquí — R^2 de test 0.7745 frente a 0.7690 de la red neuronal (RMSE de test 1.445 frente a 1.463), y se mantuvo levemente adelante durante todas las mejoras probadas sobre

la NN. Es el resultado esperable: ~130.000 filas de atributos tabulares mixtos + texto disperso es territorio clásico de árboles con gradient boosting.

Pero la NN no rinde mal: 0.769 frente a 0.775 es una brecha relativa de RMSE de apenas ~1.2%. **El hallazgo más consistente del proyecto:** lo que realmente movió la precisión en ambos modelos fue el conjunto completo de 2044 columnas (tabulares + TF-IDF) más el ajuste de hiperparámetros — no la arquitectura. La ablación solo-tabular de la NN cayó a R^2 0.623 (peor que su primera corrida completa, 0.725); el mismo bloque TF-IDF levantó a XGBoost de ~0.72 a 0.775 en un solo paso.

¿HAY OVERFITTING?

El train R^2 es más alto que el de test

MODELO	R^2 TRAIN	R^2 TEST	BRECHA
XGBoost	0.878	0.775	0.103
Red Neuronal	0.906	0.769	0.137

Sí, hay una brecha train/test — pero no es falta de regularización (ambos modelos ya aplican medidas reales contra el overfitting: `reg_alpha` / `reg_lambda` / subsample en XGBoost, dropout/weight decay/BatchNorm/early stopping en la NN). Se detecta comparando métricas train vs. val/test (arriba) y se mitiga, en parte, por cuatro causas estructurales del propio dataset:

1. Los atributos codificados por objetivo (`winery_avg_points`, etc.) filtran algo de estructura de las etiquetas de entrenamiento por construcción, pese al CV de 5 folds.
2. Ambos modelos tienen capacidad para ajustar ruido subjetivo no reproducible (el sesgo del catador solo explica ~10% de la varianza medible).
3. El bloque TF-IDF es de alta dimensión frente a las filas de entrenamiento (2000 columnas contra 83 mil filas).
4. Optuna optimizó el RMSE de validación, no la brecha train-val — una brecha más amplia se tolera mientras no cueste nada en validación.

EJEMPLO APLICADO

Predicción de un vino nuevo

Tres ejemplos sintéticos que abarcan el rango de precio/calidad, con los dos mejores modelos sobre datos nunca vistos durante el entrenamiento:

VINO	PRECIO	XGBOOST	RED NEURONAL
Merlot cotidiano (EE.UU.)	\$12	83.6	84.9
Malbec de gama media (Argentina)	\$28	86.8	87.7
Ensamblaje estilo Burdeos (Francia)	\$250	94.6	91.8

¿QUÉ MEJORARÍA CON MÁS TIEMPO O DATOS?

Modelado

- Reemplazar el bloque TF-IDF por un embedding de oraciones preentrenado (p. ej. `sentence-transformers`).
- Seguir ampliando la búsqueda de hiperparámetros de la NN, ahora que es barata — más trials, dropout por capa.
- Excluir BatchNorm/bias del weight decay — buena práctica conocida, no aplicada todavía.
- Un ensamble barato (promediar XGBoost + NN) — sus errores pueden estar decorrelacionados.

Datos

- Más catadores por vino, promediados, para reducir el piso de ruido por sesgo del catador.
- Etiquetas de sub-región consistentes en todos los países, no solo en EE.UU.
- Momento de la reseña normalizado respecto a la fecha de añada, para eliminar el sesgo de supervivencia.
- Más reseñas para categorías subrepresentadas específicamente, no solo más filas en general.
- Una segunda fuente de reseñas, para probar si el texto generaliza más allá de una sola publicación.

3.6 CONCLUSIONES

Lecciones aprendidas

XGBoost fue práctico: API tipo sklearn, sin loop manual. Sus únicas sorpresas reales:

`torch.cuda.is_available()` no sirve para detectar su soporte CUDA (usa su propio runtime), y `shap.TreeExplainer` fallaba contra el formato `base_score` de XGBoost 3.x. Su mayor ganancia vino de agregar TF-IDF, no de tunear hiperparámetros.

La red neuronal tuvo muchas más piezas:

`BatchNorm1d` falla en un último batch de tamaño 1; un bug sutil dejó de persistir la función de activación del checkpoint (`load_state_dict` no lo detecta, porque ReLU/GELU/SiLU no tienen parámetros propios). Hicieron falta tres rondas de tuning para cerrar la mayor parte de la brecha con XGBoost.

El **dataset** exigió trabajo de diseño real: un pipeline de 4 etapas (`DataCleaner` — `FeatureEngineer` — `TabularEncoder` — `TextEncoder`), y una lección sobre mantener una única fuente de verdad, tras encontrar una segunda clase de preprocesamiento huérfana y desincronizada en el código.

Síntesis final

Sobre ~130.000 reseñas de Wine Enthusiast, **XGBoost (R^2 test = 0.7745) superó levemente a la red neuronal (R^2 test = 0.7690)** — el resultado esperado para un dataset tabular de este tamaño. La diferencia entre ambos modelos fue pequeña (~1.2% de RMSE relativo); lo que realmente importó para los dos fue el conjunto de atributos (sumar el texto vectorizado) y el presupuesto de búsqueda de hiperparámetros, no la familia de modelo elegida.

El techo de ambos modelos no está en la arquitectura sino en el propio dataset: ~10% de la varianza del puntaje es ruido de catador no modelable, y varias columnas clave (`region_2`, `price`) tienen datos faltantes estructurales. Con más reseñas por vino y datos de precio/región más completos, ambos modelos probablemente se acercarían más a su límite teórico.